Basic Network Sniffer Using Python

CodeAlpha Cyber Security Internship

Prepared By

Abdul Wasay

Programming Language

Python 3.14

Library Used

Scapy

Date

July 2026

---

1. Introduction

Network traffic monitoring is one of the fundamental concepts in cybersecurity. Every device connected to a network continuously sends and receives packets containing important information. A Network Sniffer is a tool used to capture and analyze these packets in real time.

The purpose of this project is to develop a Basic Network Sniffer using Python and the Scapy library. The application captures live network packets and displays useful information such as IP addresses, transport protocols, port numbers, timestamps, and packet length. This project was completed as part of the CodeAlpha Cyber Security Internship to improve practical knowledge of packet analysis and network monitoring.

---

2. Project Objectives

The objectives of this project are:

- Capture live network packets.
- Analyze packet information.
- Display source and destination IP addresses.
- Identify transport protocols such as TCP, UDP, and ICMP.

- Display source and destination port numbers where applicable.
- Display packet timestamps and packet length.
- Understand the fundamentals of packet analysis using Python.

---

3. Tools and Technologies

The following technologies were used to develop this project:

Tool	Purpose
Python 3.14	Programming Language
Scapy	Packet Capture and Analysis
Npcap	Packet Capture Driver for Windows
Visual Studio Code	Code Editor
Windows PowerShell	Program Execution

---

4. Project Features

The Basic Network Sniffer provides the following features:

- Live packet capture
- Source IP detection
- Destination IP detection
- TCP packet identification
- UDP packet identification
- ICMP packet identification
- Source port detection
- Destination port detection
- Packet length display
- Timestamp display
- Continuous real-time monitoring
- Graceful termination using Ctrl + C

---

5. Working Principle

The application uses the Scapy library to monitor network interfaces and capture packets in real time.

For every captured packet, the program performs the following steps:

1. Capture the incoming packet.
2. Record the current timestamp.
3. Check whether the packet contains an IP header.
4. Extract the source and destination IP addresses.
5. Determine the transport protocol.
6. Display source and destination port numbers when available.
7. Display the packet size.
8. Wait for the next packet.

---

6. Code Explanation

The application is divided into the following logical components:

Packet Capture

The Scapy `sniff()` function continuously listens for packets on the selected network interface.

Packet Processing

Each packet is passed to the `process_packet()` function, which extracts useful information.

Protocol Detection

The application checks whether the packet belongs to TCP, UDP, or ICMP and displays the corresponding protocol.

Port Identification

For TCP and UDP packets, the source and destination port numbers are displayed.

Packet Information

The application displays:

- Timestamp
- Source IP Address
- Destination IP Address
- Protocol
- Source Port
- Destination Port
- Packet Length

---

7. Sample Output

Insert screenshots of your application here.

Suggested screenshots:

- Screenshot of Visual Studio Code showing the Python source code.
- Screenshot of the terminal displaying captured packets.

---

8. Challenges Faced

During the development of this project, several challenges were encountered:

- Installing Python correctly.
- Installing the Scapy library.
- Configuring Npcap on Windows.
- Understanding packet structures.
- Identifying different network protocols.
- Displaying packet information in a readable format.

These challenges were resolved through testing and configuration.

---

9. Future Improvements

The application can be enhanced with additional features such as:

- Packet filtering by protocol.
- Saving captured packets to a file.
- Exporting packets to PCAP format.
- Graphical User Interface (GUI).
- Network traffic statistics.
- Packet search functionality.
- Real-time packet logging.
- Protocol-specific filtering.

---

10. Conclusion

The Basic Network Sniffer successfully captures and analyzes live network traffic using Python and the Scapy library. The project demonstrates fundamental concepts of packet analysis, protocol identification, and network monitoring. It also provides practical experience with cybersecurity tools and strengthens understanding of how data travels across computer networks.

This project improved my Python programming skills, introduced me to packet analysis techniques, and enhanced my knowledge of cybersecurity fundamentals.